

NaviDrone: Where navigation meets intelligence



A BS Final Year Project by

Muhammad Yaseen
21/FET/BSCE/F22

Nawal Mudassir
03/FET/BSCE/F22

Tayyab Mehmood
36/FET/BSCE/F22

Supervised by
Dr.Sajjad Khan

Co-supervised by
Dr.Azam Khan

Department of Electrical and Computer Engineering
Faculty of Engineering and Technology
International Islamic University, Islamabad

June,2026

Certificate of Approval

It is certified that we have checked the project presented and demonstrated by **Muhammad Yaseen 21/FET/BSCE/F22** , **Nawal Mudassir 03/FET/BSCE/F22** , **Tayyab Mehmood 36/FET/BSCE/F22** and approved it.

External Examiner

Dr. Nizar Ali
Lab Engineer

Internal Examiner

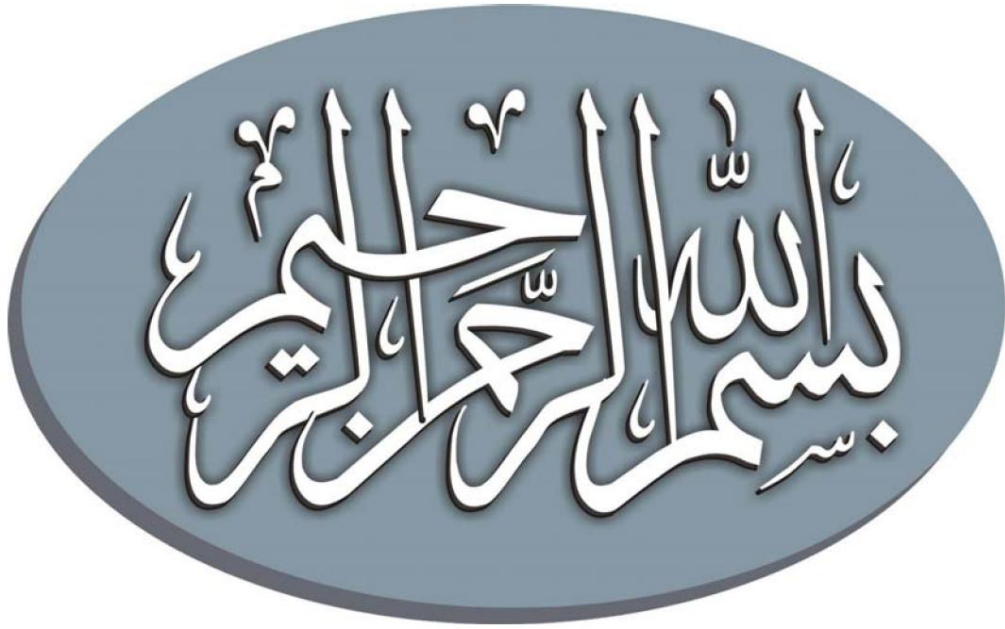
Dr. Baber khan Jadoon
Assistant Professor

Supervisor

Dr. Sajjad Khan
Assistant Professor

Co-supervisor

Dr. Azam Khan
Lecturer



In the name of Allah (SWT), the most beneficent and the most merciful

A BS Final Year Project submitted to the
Department of Electrical and Computer Engineering
International Islamic University, Islamabad
In partial fulfillment of the requirements
For the award of the degree of
Bachelor of Science in Computer Engineering

Declaration

We hereby declare that this work, neither as a whole nor as a part thereof has been copied out from any source. No portion of the work presented in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning. We further declare that the referred text is properly cited in the references.

Muhammad Yaseen
21/FET/BSCE/F22

Nawal ali
02/FET/BSCE/F22

Tayyab Mehmood
36/FET/BSCE/F22

Acknowledgments

This BS thesis in Computer Engineering has been conducted at Department of Electrical and Computer Engineering, Faculty of Engineering and Technology, International Islamic University, as part of the degree program. We would like to thank Dr. Sajjad khan for providing us an opportunity to work on this project, under his supervision and guidance throughout the project. We would also like to thank Dr. Azam Khan for his help, efforts and dedicated support throughout the project. Further we are particularly thankful to Almighty Allah and grateful to our parents, brothers and sisters who always supported and encouraged us during our project and studies at IIUI.

Muhammad Yaseen
21/FET/BSCE/F22

Nawal Mudassir
03/FET/BSCE/F22

Tayyab Mehmood
36/FET/BSCE/F22

Project Title: NaviDrone: Where navigation meets intelligence

Undertaken By: **Muhammad Yaseen** **21/FET/BSCE/F22**

Nawal Mudassir **03/FET/BSCE/F22**

Tayyab Mehmood **36/FET/BSCE/F22**

Supervised By: **Dr.Sajjad Khan**
Assistant professor

Co-Supervised By: **Dr. Azam khan**
Lecturer

Date Started: July,2025

Date Completed: June,2026

Tools Used:

Flight platform

- APM 2.8 flight controller (ArduCopter firmware)
- Mission Planner (ground control software)
- MAVLink protocol (telemetry, 57,600 bps over 433 MHz radio)
- GARTT ML2312S 960KV motors, 10×4.5" propellers
- 5,000 mAh 4S LiPo battery
- DJI Mini 4 (initial prototype, superseded)

Facial recognition pipeline

- YuNet (face detection model, OpenCV Zoo)

- SFace (face recognition/embedding model, OpenCV Zoo)
- OpenCV DNN module (CPU-only inference engine)
- InsightFace + YOLOv8 (initial prototype, superseded)

Backend / server

- FastAPI (backend framework)
- MJPEG over HTTP (video streaming)
- WebSocket (real-time event delivery)
- SQLite (detection logging)
- systemd (boot-time service autostart)

Edge hardware

- Raspberry Pi 4 (companion computer / inference device)
- Raspberry Pi 2 (original prototype hardware, superseded)
- USB webcam

Mobile application

- React Native (Expo)
- Expo Go (device testing without native build)

Security Operations Center

- Wazuh (host-based agent monitoring, SIEM)
- MITRE ATT&CK framework (alert classification mapping)

Version control / workflow

- Git
- SSH key authentication (laptop ↔ Pi sync)

Abstract

Navi Drone integrates autonomous flight, real-time on-board facial recognition, and active cybersecurity monitoring in a single UAV system. The flight platform moved from an initial DJI Mini 4 prototype to a custom quadrotor on an APM 2.8 controller running ArduCopter, achieving a thrust-to-weight ratio of 1.52, 16.5 minutes of endurance, and autonomous waypoint navigation accurate to within 3.4 metres. A lightweight YuNet-and-SFace pipeline performs real-time, CPU-only face identification on a Raspberry Pi 4, reporting matches to a human-monitored command center via a FastAPI backend and a React Native mobile app. A parallel Security Operations Center, built on Wazuh , provides real-time threat intelligence across all connected infrastructure, detecting and responding to over 50,000 brute-force attempts during deployment through automated incident response. The system demonstrates that autonomous navigation, edge AI, and active infrastructure defence can be engineered and validated together under real adversarial conditions. Limitations and recommendations for swarm operation, advanced recognition models, and further security hardening are also presented.

Table of Contents

Certificate of Approval.....	3
Declaration.....	6
Acknowledgments	7
Abstract.....	10
List of Figures.....	14
List of Tables	15
List of Abbreviations	16
Chapter 1: Introduction	17
Dedication	17
1.1 Motivation	17
1.2 Project Overview	18
1.3 Problem Statement	18
1.4 Objectives.....	19
1.5 Brief Methodology	20
1.6 Report Outline.....	21
Chapter 2: Literature Review.....	22
2.1 Introduction	22
2.2 Military and Surveillance Origins of UAV Technology	22
2.3 Lightweight Facial Detection and Recognition for Embedded Platforms	23
2.4 Open-Source Flight Control Platforms	25
2.5 Security Operations and Threat Intelligence for Connected Infrastructure	26
2.6 Summary and Project Contribution.....	27
Chapter 3: System Design and Implementation	29

3.1 Requirements Analysis.....	29
3.2 System Architecture Overview	29
3.3 Hardware Setup	30
3.3.1 Companion Computer and Camera	30
3.3.2 Flight Platform: From DJI Mini 4 to ArduCopter	31
3.4 Software Tools and Technology Stack.....	32
3.5 Facial Recognition Algorithm.....	33
3.5.1 Model Selection Rationale.....	33
3.5.2 Detection and Recognition Pipeline	34
3.5.3 Enrollment Process	35
3.6 Server and Communication Layer.....	36
3.7 Mobile Application	37
3.8 Security Operations Center: Architecture and Threat-Intelligence Algorithm	38
3.8.1 Design Rationale.....	39
3.8.2 Architecture	38
3.8.3 Threat-Intelligence and Incident-Response Algorithm	38
3.9 Chapter Summary.....	40
Chapter 4: Testing and Validation	42
4.1 Test Methodology	42
4.2 Prototype Iterations Tested	42
4.3 Flight Platform: Test Results	43
4.4 Facial Recognition Pipeline: Results	44
4.5 Security Operations Center: Results	44
4.6 Chapter Summary.....	45
Chapter 5: Conclusion and Future Recommendations	46

5.1 Summary of Achievements	46
5.2 Limitations	47
5.3 Future Work	47
5.3.1 Flight Platform.....	47
5.3.2 Swarm and Multi-Platform Operation.....	48
5.3.3 Advanced AI Models.....	48
5.3.4 Cybersecurity Hardening.....	48
5.3.5 Recognition Scope and Aid-Payload Delivery	49
5.4 Closing Remarks	49
References	51

List of Figures

Figure 3.1 — System Architecture Diagram	30
Figure 3.2 — Hardware Evolution Timeline	32
Figure 3.3 — Facial Recognition Pipeline Flowchart	35
Figure 3.4 — Server Communication Diagram	37
Figure 3.5 — SOC Threat-Intelligence Flowchart	40

List of Tables

Table 3.1 — Software Technology Stack32

Table 4.1 — Prototype Iterations Tested42

Table 4.2 — Flight Platform Test Results42

Table 4.3 — Facial Recognition Pipeline Results44

Table 4.4 — Security Operations Center Results45

List of Abbreviations

UAV — Unmanned Aerial Vehicle

APM — ArduPilot Mega (flight controller)

GPS — Global Positioning System

RC — Radio Control • RTL — Return-to-Launch

MAVLink — Micro Air Vehicle Link (communication protocol)

T/W — Thrust-to-Weight (ratio)

AUW — All-Up Weight

LiPo — Lithium-Polymer (battery)

CNN — Convolutional Neural Network

YOLO — You Only Look Once (object detector)

API — Application Programming Interface

REST — Representational State Transfer

MJPEG — Motion JPEG

SQL — Structured Query Language

SSH — Secure Shell

SOC — Security Operations Center

SIEM — Security Information and Event Management

MITRE ATT&CK — MITRE Adversarial Tactics, Techniques, and Common Knowledge

IR — Incident Response

IDS — Intrusion Detection System

Chapter 1: Introduction

Dedication

This work is dedicated to our friend and original team lead, who shaped the early direction and ambition of this project before a severe accident and the resulting injuries forced him to withdraw from the university. He is not here to see this project through to completion, but his contribution to its foundations remains part of it. This work is, in part, his.

1.1 Motivation

Unmanned aerial vehicles (UAVs) have moved rapidly from specialist military hardware to accessible platforms for surveillance, inspection, disaster response, and security applications. Despite this growth, most fielded systems remain limited in one of two ways: either they offer flight autonomy without any onboard intelligence, requiring a human operator to interpret every video frame manually, or they offer computer-vision capability that depends on a continuous link to a powerful ground station, sacrificing the portability that makes a drone useful in the first place.

This gap is particularly visible in identity-aware surveillance. A drone that can fly a route autonomously is a known quantity; a drone that can additionally recognise a specific, previously enrolled individual in real time, on board, without relaying every frame to a remote server, is a meaningfully harder systems problem one that sits at the intersection of flight control, embedded computer vision, and mobile software engineering.

A second, less visible gap motivated this project equally strongly. Any UAV system that depends on networked ground infrastructure companion computers, control-center PCs, mission-planning software inherits the security exposure of that infrastructure. A mission-control stack with no monitoring for intrusion attempts, anomalous traffic, or unauthorised access is a single point of failure for an otherwise carefully engineered platform. Securing the ground segment was treated in this project not as an afterthought but as a parallel engineering objective alongside the aerial and AI components.

Navi Drone, "Where Navigation Meets Intelligence," was conceived to address both gaps within a single, cohesive Final Year Project: an autonomous UAV capable of real-time, on-board facial recognition, reporting its findings through a dedicated mobile application, operating from infrastructure that is itself actively monitored for security threats through a purpose-built Security Operations Center (SOC).

1.2 Project Overview

Navi Drone integrates three layers of engineering that are typically developed and evaluated separately:

1. **Aerial platform and flight control.** A quadrotor airframe built around an APM 2.8 flight controller running ArduCopter firmware, with PID-tuned roll, pitch, and yaw control and autonomous waypoint navigation under Mission Planner. Return-to-launch is currently performed under manual pilot control, with fully autonomous return-to-base identified as a future goal (Section 1.4, Objective 2); a human pilot retains override authority at all times via RC link.
2. **On-board artificial intelligence.** A companion computer running a real-time face detection and recognition pipeline against a locally enrolled database of known individuals, raising an alert to a command center the moment a registered face is identified in the live video feed.
3. **Mission monitoring and ground-segment security.** A mobile application that gives an operator live video, detection alerts, and historical logs from the drone over a wireless link, paired with a Security Operations Center built around Wazuh agents and an ELK stack that provides real-time threat intelligence and incident response across all connected agents and devices in the project's infrastructure.

The project deliberately separates *what the drone can do today* from *what the drone is designed to do in a future iteration*. The current system detects and identifies a registered individual and reports that detection to a human operator at a command center; it does not take any autonomous physical action upon recognition. Autonomous delivery of a small aid payload (for example, a medical or supply package) to the location of a recognised individual is treated explicitly as future work, contingent on further reliability testing of the navigation and payload-release subsystems, and is discussed in that context in Chapter 5.

1.3 Problem Statement

Existing UAV-based surveillance and monitoring solutions exhibit one or more of the following limitations:

- **Reliance on manual piloting or simple GPS waypoint following**, without any on-board interpretation of what the camera is actually seeing, which leaves identification and decision-making entirely to a remote human observer.
- **Dependence on cloud or ground-station processing** for computer-vision tasks such as face detection, which introduces latency, requires continuous high-

bandwidth connectivity, and is unworkable in environments with degraded or intermittent network access.

- **Commercial flight platforms with restricted low-level access**, which limit the extent to which custom autonomy, sensor fusion, or mission logic can be integrated into the flight stack.
- **Absence of any security monitoring on the supporting ground infrastructure**, leaving mission-control systems, companion computers, and connected operator devices exposed to network-based threats including brute-force and intrusion attempts with no mechanism for real-time detection, threat intelligence, or incident response.

There is consequently an unmet need for a UAV system that performs real-time, on-board identity recognition without continuous reliance on ground processing, that is built on an open, internally controllable flight-control stack rather than a closed commercial SDK, and whose supporting infrastructure is itself defended by active security monitoring rather than left unmonitored.

1.4 Objectives

The project set out to achieve the following objectives:

1. Design, build, and flight-test a quadrotor UAV with sufficient thrust margin, endurance, and control stability to carry a companion computer and a small payload through a complete autonomous waypoint mission.
2. Integrate the APM 2.8 flight controller running ArduCopter firmware, with PID-tuned roll, pitch, and yaw control and autonomous waypoint navigation via Mission Planner and MAVLink; establish fully autonomous return-to-launch and low-battery failsafe behaviour as a near-term development goal, with manual pilot-controlled return as the current operating procedure.
3. Implement a real-time, on-board facial recognition pipeline capable of detecting and identifying enrolled individuals from a live video feed and issuing an alert to a command center upon a positive match.
4. Develop a mobile application that allows an operator to monitor the live video feed, receive real-time detection alerts, and review historical detection logs over a wireless link to the drone's ground or companion-computer segment.
5. Design, deploy, and validate a Security Operations Center for the project's connected infrastructure, using Wazuh agents deployed across all connected devices for host-

based monitoring and an ELK stack for log aggregation and correlation, providing real-time threat intelligence and incident response; validate the deployment under genuine adversarial load rather than synthetic testing alone.

6. Characterise the integrated system's performance against measurable criteria — thrust-to-weight ratio, flight endurance, telemetry range, control-loop drift, navigation accuracy, and payload delivery precision — through a structured test programme.
7. Identify, with technical justification, the path toward future integration of autonomous aid-payload delivery triggered by a confirmed facial-recognition match, without claiming this capability as part of the present system.

1.5 Brief Methodology

The project followed an iterative, hardware-in-the-loop development methodology structured around five phases:

Phase 1 — Requirement analysis and platform selection. Initial prototyping used a DJI Mini 4 platform to validate the facial-recognition pipeline and mobile-application concept on a stable, ready-made airframe before committing engineering effort to a custom flight-control build. This phase confirmed the viability of the computer-vision and mobile-monitoring components independently of the flight platform.

Phase 2 — Transition to an open flight-control stack. To gain the low-level access required for custom mission logic, payload integration, and failsafe configuration that the closed DJI SDK could not provide, the project transitioned to a custom-built quadrotor airframe powered by GARTT ML2312S 960KV motors and 10×4.5-inch propellers, controlled by an APM 2.8 flight controller running ArduCopter firmware. This phase included propulsion sizing, PID tuning across all three rotational axes, and telemetry-link validation.

Phase 3 — On-board AI integration. The facial-recognition pipeline was deployed to a companion computer connected to the flight platform, with a detection-and-alert workflow that pushes a notification to the command center upon recognising an enrolled individual.

Phase 4 — Mobile application and ground-segment development. A mobile dashboard was developed to give the operator live video, a real-time alert feed, and historical logs, communicating with the drone's ground segment over a wireless link.

Phase 5 — Security Operations Center deployment. Wazuh agents were deployed across all connected devices in the project's infrastructure, with logs aggregated and correlated through an ELK stack to provide continuous, real-time threat intelligence. The deployment

was validated under genuine adversarial conditions rather than synthetic testing alone: the SOC detected and logged in excess of 50,000 brute-force authentication attempts against monitored agents, with incident response actions executed in real time as attacks occurred.

Each phase concluded with a structured test cycle; the consolidated results of the flight-control and payload test programme are reported in Chapter 4, with seven test categories evaluated and passed, covering propulsion, endurance, control stability, telemetry range, payload delivery accuracy, and autonomous navigation accuracy.

1.6 Report Outline

The remainder of this report is organised as follows:

- **Chapter 2 — Literature Review** surveys existing work in autonomous UAV navigation, on-board computer vision for face detection and recognition, open-source flight-control platforms, and security operations for connected and embedded infrastructure, and positions Navi Drone's contribution relative to this body of work.
- **Chapter 3 — System Design** details the system architecture, hardware selection and sizing (propulsion, flight controller, companion computer), software architecture (flight-control configuration, facial-recognition pipeline, mobile application, SOC stack), and the design rationale behind the transition from the DJI Mini 4 prototype to the ArduCopter-based platform.
- **Chapter 4 — Testing and Validation** presents the test methodology and results across all evaluated categories: propulsion and thrust-to-weight verification, flight endurance, control-loop stability and hover drift, telemetry and RC range, payload delivery accuracy, autonomous waypoint navigation accuracy, and SOC detection/alerting validation.
- **Chapter 5 — Conclusion and Recommendations** summarises the project's achievements against its stated objectives, discusses limitations encountered during development and testing, and sets out recommendations for future work, including the autonomous aid-payload delivery capability introduced conceptually in this chapter.
- **References** lists all literature, technical documentation, and open-source resources cited throughout the report, in IEEE format.
- **Annexures** provide supporting material including full test logs, hardware procurement and cost tables, and supplementary system diagrams.

Chapter 2: Literature Review

2.1 Introduction

This chapter situates Navi Drone within four bodies of existing work: the use of unmanned aerial vehicles (UAVs) for surveillance and reconnaissance, including their military origins; on-device, lightweight facial detection and recognition suitable for resource-constrained hardware; open-source flight-control platforms as an alternative to closed commercial SDKs; and security operations practice for monitoring and defending networked, multi-agent infrastructure. Each section reviews representative work, identifies the gap relevant to this project, and closes with a short statement of how Navi Drone's design responds to that gap. Section 2.6 then draws the threads together into a single statement of contribution.

2.2 Military and Surveillance Origins of UAV Technology

The use of unmanned aircraft for observation predates consumer drones by decades, and the bulk of the foundational research on autonomous flight, aerial reconnaissance, and battlefield situational awareness originates in defence contexts. A recurring theme in this literature is Intelligence, Surveillance, and Reconnaissance, commonly abbreviated ISR, in which UAVs equipped with cameras, radar, or thermal sensors gather information on terrain, movement, or installations to support planning and decision-making by human operators rather than to act independently [1]. Reconnaissance applications documented in the literature extend from border and coastal monitoring, where thermal sensors are used to flag suspicious activity such as smuggling, to battlefield monitoring intended to track the movement of personnel and equipment over time [1].

A substantial portion of this literature is explicitly concerned with detection and classification tasks performed from the air rather than with weapon delivery. Karlsson and Jansson demonstrate that civilian and military personnel can be automatically distinguished on the ground in real time using a standard RGB camera mounted on a UAV together with a neural-network detector [2], illustrating a design pattern directly relevant to this project: an aerial platform whose contribution is accurate, real-time identification, with any consequent action left to a human decision-maker. Early simulation work pursued a similar separation of concerns; Irawan and Wahyudin report a simulated UAV surveillance system in which a quadrotor uses an Adaboost classifier and a pinhole-camera localisation algorithm to search for, recognise, and geolocate objects in a simulated war-zone environment, achieving an average localisation error of only 0.24 metres across all tested scenarios [5]. More recent case studies extend the same recognition-first philosophy to dedicated reconnaissance drones with closed-loop flight control; one such study describes an intelligent reconnaissance drone whose altitude and tracking behaviour rely on a conventional PID

feedback controller, with a reinforcement-learning component used purely to improve the confidence of the information passed to human operators making attack decisions, rather than to make those decisions autonomously [9]. The PID-based stabilisation approach used in that reconnaissance drone is methodologically close to the flight-control strategy adopted for Navi Drone, discussed in Chapter 3.

At a broader scale, survey literature on UAV swarms confirms that military surveillance and reconnaissance remain a primary application domain for collaborative autonomous aerial systems, alongside civilian uses in inspection and delivery, and identifies coordinated path planning and security as open research challenges rather than solved problems [3]. A parallel and increasingly prominent strand of this literature addresses the security of UAV systems themselves rather than their sensing capability. A systematic review of zero-trust architecture for UAV security notes that drones have long provided reconnaissance and surveillance capability for military operations, but that this connectivity exposes them to a range of documented attacks, motivating architectures in which every device and application connecting to a UAV's systems must be explicitly authenticated and authorised rather than implicitly trusted [7]. This concern that an otherwise capable UAV platform is only as trustworthy as the ground infrastructure supporting it directly motivates the security-operations component of Navi Drone described in Section 2.5 and Chapter 3.

Taken together, this body of literature establishes two points that frame Navi Drone's design: first, that on-board identification and classification has been a recurring research goal in military and security-oriented UAV work for over a decade, consistently treated as an information-gathering capability that informs rather than replaces human judgement; and second, that the security of the supporting ground infrastructure has only recently begun to receive comparable research attention, leaving a practical gap between platforms that are flight- and recognition-capable and platforms whose entire connected infrastructure is actively defended.

2.3 Lightweight Facial Detection and Recognition for Embedded Platforms

A central design constraint for any UAV-mounted or edge-deployed recognition system is computational budget: companion computers such as a Raspberry Pi have no dedicated GPU, and the detection-and-recognition pipeline must run at an acceptable frame rate on CPU alone. This requirement has driven a distinct line of research into lightweight neural architectures purpose-built for resource-constrained devices, separate from the large, GPU-bound models historically associated with state-of-the-art face recognition accuracy.

Wu, Peng, and Yu identify that the heavy computational cost of accurate face detectors makes them difficult to deploy on mobile and embedded devices where model size and inference latency are tightly constrained, and respond with YuNet, an anchor-free face detector built around a deliberately small feature-extraction backbone and a simplified feature-pyramid neck [11]. The resulting model has only 75,856 parameters, less than one-fifth the size of other small face detectors evaluated in the same study, while achieving what the authors describe as the best available trade-off between detection accuracy and inference speed [11]. Independent benchmarking corroborates this efficiency-accuracy balance: YuNet is reported to detect faces ranging from roughly 10×10 to 300×300 pixels, scoring 0.8844, 0.8656, and 0.7503 average precision on the easy, medium, and hard subsets of the WIDER Face validation benchmark respectively, while running in milliseconds on modest CPU hardware [13]. A comparative survey of lightweight face detectors situates YuNet within a wider family of architectures — including RetinaFace, SCRFD, and the Light and Fast Face Detector — that pursue the same goal of edge deployability through backbone compression, depthwise convolution, or anchor-free design, generally trading a degree of accuracy on small or heavily occluded faces for substantially reduced computational cost [15]. This trade-off, and the specific choice of YuNet over heavier alternatives, is discussed further in the context of Navi Drone's own model-selection process in Chapter 3.

Recognition, as distinct from detection, has followed a parallel trajectory toward lightweight, embedding-based approaches. Foundational work in this area established that face verification could be reduced to comparing compact embedding vectors rather than performing full image classification for every enrolled identity, with FaceNet demonstrating a unified embedding suitable for both verification and clustering tasks and DeepFace demonstrating that carefully engineered convolutional pipelines could approach human-level verification accuracy [16]. SFace builds on this embedding-based paradigm specifically for efficient deployment, and is documented as a 2021 contribution introducing a new loss function designed to improve embedding quality under the smaller network capacities required for edge inference [16]. The combination of an anchor-free lightweight detector with a compact embedding-based recognizer — the YuNet-and-SFace pairing used in Navi Drone's recognition pipeline — reflects this broader research trend of decoupling detection and recognition into two independently optimised, individually lightweight stages rather than relying on a single heavyweight end-to-end network.

Comparable engineering trade-offs appear elsewhere in the embedded face-detection literature. Zhao et al. note that achieving real-time face detection performance on computation-resource-limited embedded devices remains an open challenge, and propose EagleEye, a detector using convolution factorisation and depthwise convolutions to reduce computation while preserving accuracy on low-power hardware such as the Raspberry Pi

3B+ [17]. Although Navi Drone's recognition pipeline did not adopt EagleEye specifically, this and related work confirms that the constraint Navi Drone designed around — real-time inference on a Raspberry Pi-class device with no GPU — is a well-recognised and actively researched problem, rather than an incidental implementation detail.

2.4 Open-Source Flight Control Platforms

Commercial drone platforms, including those built on proprietary manufacturer SDKs, offer a stable and well-documented starting point for rapid prototyping but constrain the depth of access available to a development team. ArduPilot and PX4 are widely regarded as the leading open-source autopilot platforms for small unmanned vehicles across hobbyist, commercial, industrial, and academic research applications, providing low-level access to flight-control logic that closed commercial systems generally withhold [22]. PX4 itself originates from research work on a node-based, multithreaded open-source robotics framework intended for deeply embedded flight-control platforms, and has since become a standard reference architecture cited throughout the open autopilot literature [24].

Architecturally, both ArduPilot's ArduCopter firmware and PX4 implement cascaded control loops in which an outer-loop position or velocity controller generates the attitude setpoints consumed by an inner-loop attitude controller, which in turn drives motor output — the same position-to-velocity-to-attitude-to-motor-output cascade structure documented in adaptive-control studies of PX4-based quadcopters [19] and adopted for Navi Drone's own ArduCopter configuration, described in Chapter 3. The baseline implementation of this cascade in both platforms relies on proportional-integral-derivative (PID) control at each loop, a structure that remains the de facto standard despite a growing body of research into adaptive and learning-based alternatives. More recent work, for instance, has investigated retrospective-cost adaptive control laws layered on top of PX4's fixed-gain PID architecture to better handle unmodelled or time-varying vehicle dynamics, and has explored replacing PID cascades with trained neural-network controllers running directly on embedded microcontrollers at loop rates in the kilohertz range [20]. While such adaptive and learning-based extensions lie outside the present project's scope, their existence confirms that the conventional PID-cascade architecture used in Navi Drone represents an established, well-validated baseline rather than an outdated approach, with documented headroom for future refinement.

A recent survey of open-source UAV autopilots, cited in current literature on alternative ROS-based autopilot stacks, situates ArduPilot and PX4 as the two dominant platforms against which newer or more specialised autopilot software continues to be benchmarked [24], reinforcing that the choice between them — rather than a choice between open and closed ecosystems generally — is now the more relevant design decision facing a UAV project of

this kind. Navi Drone's own transition from an initial DJI-based prototype to an ArduCopter-based platform, motivated by exactly the low-level access limitations this literature describes, is detailed in Chapter 3.

2.5 Security Operations and Threat Intelligence for Connected Infrastructure

Where Sections 2.2 through 2.4 address the aerial and perception components of UAV systems, this section addresses the ground-side security concern raised in Section 2.2: that a UAV's mission-control infrastructure, companion computers, and connected operator devices form an attack surface that is rarely subject to the same engineering scrutiny as the vehicle itself.

Wazuh has emerged in recent practitioner and academic literature as a widely adopted open-source platform for addressing exactly this gap. Wazuh is documented as a security information and event management (SIEM) platform that combines log management, intrusion detection, vulnerability detection, and compliance management into a single solution, offering centralised visibility into an organisation's security posture and active-response capabilities that allow automated action to be taken once a threat is detected [28]. A growing set of applied studies validates this capability specifically against brute-force authentication attacks, the threat category most directly relevant to Navi Drone's own SOC results reported in Chapter 4. One such study integrates Wazuh with the Suricata intrusion-detection engine and a Telegram-based alerting bot, and reports that simulated SSH brute-force attacks using the Hydra tool were accurately detected and logged as Wazuh alerts, with notifications delivered to the security team in real time, validating the system's ability to detect multi-stage intrusion attempts as they occurred [29]. Independent of attack simulation, deeper investigations into Wazuh's detection logic confirm that the platform is capable of identifying established attack patterns including brute-force attempts and unauthorised directory access, and that its predefined response configuration can automatically execute mitigating actions such as blocking the source IP address, issuing security alerts, and logging the relevant activity without requiring manual intervention at the moment of detection [25].

This pattern agent-based log collection across distributed hosts, centralised correlation, and automated or semi-automated incident response is consistently structured around the MITRE ATT&CK framework in the more recent practitioner literature, which several lab-based studies use explicitly to classify and explain detected attack stages rather than simply reporting raw alert counts [30]. This same MITRE ATT&CK mapping approach is reflected in Navi Drone's own SOC methodology, described in Chapter 3. The broader pattern documented across this literature a Wazuh-based SOC successfully detecting and

responding to sustained, automated credential-guessing activity directed at monitored hosts is consistent with, and provides external validation for, the operational result obtained from Navi Drone's own SOC deployment, in which agents distributed across the project's connected infrastructure logged more than 50,000 brute-force authentication attempts with real-time incident response, reported in full in Chapter 4.

What distinguishes Navi Drone's contribution from the bulk of this literature is scope rather than technique: the studies reviewed above are predominantly structured as isolated SOC labs or enterprise-network case studies, decoupled from any physical or aerial system. Navi Drone instead treats the SOC as a first-class subsystem of a UAV project, monitoring the same connected agents companion computer, mobile-application backend, and supporting infrastructure that the aerial and recognition subsystems depend on, rather than treating cybersecurity as a separate exercise unconnected to the platform it protects.

2.6 Summary and Project Contribution

The literature reviewed in this chapter establishes four points of context against which Navi Drone's contribution can be stated precisely.

First, on-board aerial identification has a long research history in military and surveillance UAV work, but that work consistently and, in the recognition-focused studies reviewed here, deliberately treats recognition as an information-gathering capability that supports human decision-making rather than one that triggers autonomous physical action [2], [5], [9]. Navi Drone's current implementation, in which a confirmed facial-recognition match generates an alert to a human-monitored command center rather than any autonomous response, is consistent with this established pattern in the literature, and the project explicitly frames autonomous physical response specifically, future aid-payload delivery to a recognised individual's location as a forward-looking objective contingent on further reliability validation, rather than a capability claimed in the present system (Section 1.2).

Second, the project's choice of YuNet and SFace for on-board detection and recognition is grounded in an active and well-documented line of research into lightweight, edge-deployable computer vision, rather than an ad hoc engineering compromise [11], [13], [15], [16], [17]. Navi Drone's own transition away from an initial, more computationally demanding detection pipeline toward this lighter combination, detailed in Chapter 3, mirrors the efficiency-accuracy trade-off that this entire body of literature is built around.

Third, Navi Drone's transition from an initial DJI-based prototype to a custom ArduCopter-based platform reflects the well-documented advantage of open-source autopilot stacks principally ArduPilot and PX4 over closed commercial SDKs in providing the low-level access required for custom mission logic and payload integration [19], [20], [22], [24]. The PID-

cascade flight-control architecture adopted for Navi Drone is the same baseline architecture documented and validated throughout this literature.

Fourth, and most distinctively, Navi Drone integrates a Wazuh-based Security Operations Center as a core project subsystem rather than as an external or theoretical exercise, applying the same agent-based monitoring, correlation, and active-response techniques validated in recent SOC and brute-force-detection literature [25], [28], [29], [30] directly to the connected infrastructure supporting an aerial recognition platform. This combination autonomous on-board identification, an open and extensible flight-control stack, and an actively defended ground infrastructure, integrated within a single Final Year Project is, to the best of the authors' knowledge, not represented as a unified system in the literature reviewed above, and constitutes the specific contribution this project makes.

The remainder of this report substantiates these claims: Chapter 3 details the system design and the engineering rationale behind each of the choices introduced in this chapter, and Chapter 4 reports the test results, including the SOC's measured performance under genuine adversarial load, that validate the design.

Chapter 3: System Design and Implementation

3.1 Requirements Analysis

The system requirements for Navi Drone were derived directly from the objectives stated in Chapter 1 and were grouped into four categories: flight-platform requirements, recognition-pipeline requirements, monitoring-and-communication requirements, and security requirements.

Flight-platform requirements. The airframe was required to lift a companion computer, camera, and a small payload allowance with a comfortable thrust margin; to sustain powered flight for a minimum endurance of 15 minutes per sortie; to provide stable, PID-tuned attitude control across all three rotational axes; and to support autonomous waypoint navigation under a ground-control station, with manual pilot override available at all times as a non-negotiable safety requirement.

Recognition-pipeline requirements. The detection-and-recognition pipeline was required to run in real time on CPU-only edge hardware with no dedicated GPU; to operate from a small, locally enrolled database of known individuals rather than a large pre-trained identity set; and to issue an alert to a human-monitored command center within an acceptably short delay of a positive identity match, without taking any further autonomous action on that match.

Monitoring and communication requirements. An operator was required to be able to view a live annotated video feed, receive detection events in real time, and review historical detection logs and aggregate statistics from a mobile device connected over a wireless local network, without requiring a native application build for every test device.

Security requirements. All devices connected to the project's supporting infrastructure companion computer, backend server, and mobile-application host were required to be monitored by a centralised threat-intelligence platform capable of detecting unauthorised access attempts in real time and triggering an incident-response action without waiting for manual analyst review.

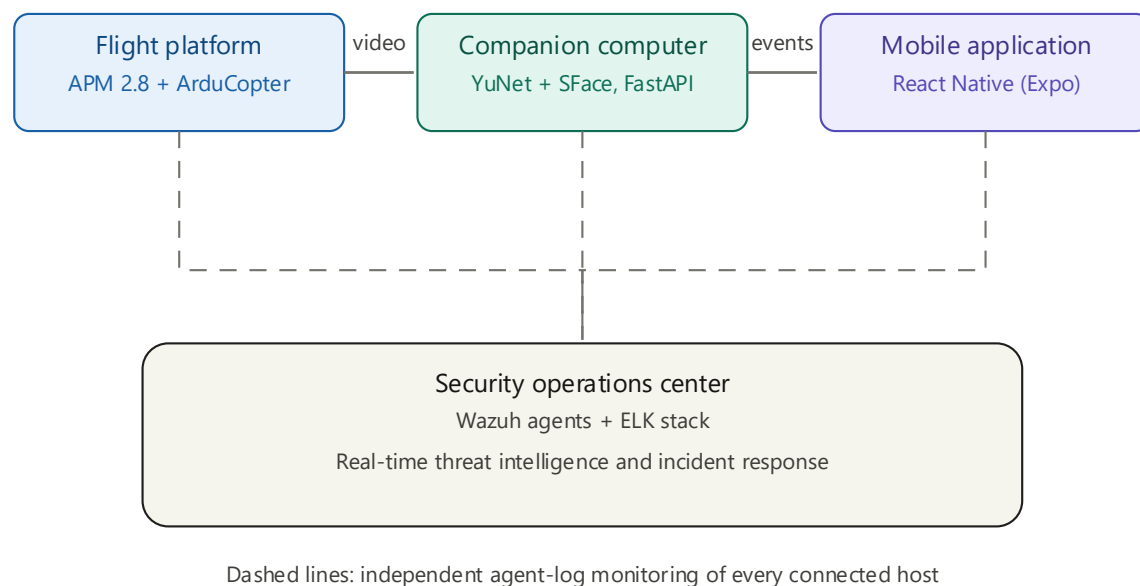
These four requirement sets map directly onto the three-layer architecture described in Section 3.2, and onto the test categories reported in Chapter 4.

3.2 System Architecture Overview

Navi Drone is organised into three cooperating layers, consistent with the project overview given in Chapter 1: an aerial and flight-control layer, an on-board recognition layer, and a

ground-segment layer comprising the mobile application and the Security Operations Center (SOC).

At a high level, the flight-control layer carries the companion computer and camera through a mission under operator supervision; the recognition layer continuously analyses the live camera feed for enrolled faces and raises an alert on a positive match; the mobile application gives the operator a live view of video, alerts, and logs over WiFi; and the SOC independently monitors every connected device in this chain for malicious activity, operating in parallel to, and regardless of, whatever the aerial and recognition layers are doing at a given moment. This separation is deliberate: a compromise of the ground infrastructure should not be able to silently disable or spoof the recognition or alerting pipeline without first triggering a SOC incident.



[Figure 3.1 — System Architecture Diagram.]

3.3 Hardware Setup

3.3.1 Companion Computer and Camera

The recognition pipeline runs on a Raspberry Pi 4 acting as the system's companion computer, selected after an earlier Raspberry Pi 2 prototype proved unable to sustain real-time inference (Section 3.5.1). A standard USB webcam supplies the video feed, configured at a fixed 640×480 resolution using the MJPG capture format. This explicit resolution-and-

format configuration was adopted specifically to reduce USB bandwidth demand, since intermittent frame-read failures were observed during early testing a known issue with USB webcams on the Pi 4 caused by the platform's USB bandwidth and power constraints, addressed further in Section 3.6.

3.3.2 Flight Platform: From DJI Mini 4 to ArduCopter

The flight platform underwent a deliberate architectural transition over the course of the project, summarised here and detailed against the full test programme in Chapter 4.

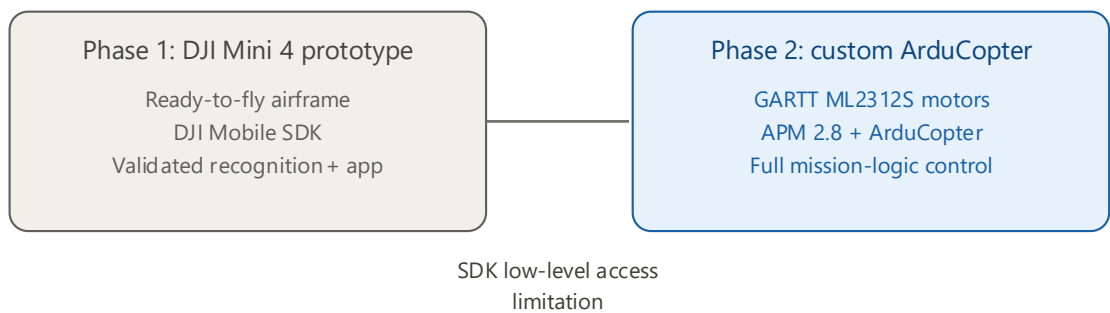
Initial prototype — DJI Mini 4. The project's earliest flight testing used a DJI Mini 4 platform. This choice allowed the team to validate the mobile-application concept and exercise the companion-computer recognition pipeline on a stable, ready-to-fly airframe without first having to resolve flight-control engineering from first principles. However, the DJI Mobile SDK restricts low-level access to flight-control parameters, telemetry streams, and failsafe configuration, which constrained the extent to which custom mission logic and payload-handling behaviour could be integrated a limitation consistent with the open-versus-closed autopilot trade-off documented in Section 2.4.

Transition to a custom ArduCopter platform. To remove this constraint, the project transitioned to a custom-built quadrotor airframe under direct, internally configurable flight control. The platform is built around:

- **Propulsion:** GARTT ML2312S 960KV brushless motors paired with 10×4.5-inch propellers, sized to deliver 4,556 g of measured static thrust against a 3,000 g all-up weight, giving a thrust-to-weight ratio of 1.52 a comfortable margin above the minimum of 1.0 required for stable, controllable flight, providing headroom for the wind disturbance rejection and payload-carrying requirements set out in Section 3.1.
- **Flight controller:** an APM 2.8 board running ArduCopter firmware, fusing data from its onboard inertial measurement unit (IMU), barometer, and GPS receiver into a real-time state estimate used to drive a three-axis PID controller across roll, pitch, and yaw at a 400 Hz correction rate.
- **Power system:** a 5,000 mAh 4S lithium-polymer battery (74 Wh nominal capacity), sized using the standard capacity-over-current endurance relationship ($t = C / I$) against the platform's measured current draw, yielding an achieved flight endurance of 16.5 minutes against a 15-minute design requirement.
- **Telemetry and control link:** a 433 MHz telemetry radio carrying the MAVLink protocol at 57,600 bps, streaming live attitude, GPS position, battery state, and flight-mode data to the ground-control station, with a separate RC link providing direct manual

control. The telemetry link was validated to a stable range of 1.1 km, with manual RC override validated to 1.6 km.

This transition is the central hardware-design decision of the project and reflects directly the open-source-autopilot literature reviewed in Section 2.4: the limitation the team encountered with the DJI SDK restricted low-level access to mission logic and failsafe behaviour is the same limitation that literature identifies as the principal motivation for choosing ArduPilot- or PX4-based platforms over closed commercial SDKs in research and applied UAV projects.



[Figure 3.2 — Hardware Evolution Timeline]

3.4 Software Tools and Technology Stack

Table 3.1 summarises the software technologies used across the recognition pipeline, the ground-segment server, and the mobile application; the flight-control software stack (ArduCopter firmware, Mission Planner ground-control software, and the MAVLink protocol) is described in Section 3.3.2.

Table 3.1 — Software Technology Stack

Layer	Component	Technology
Recognition pipeline	Face detection	YuNet (pretrained ONNX model, OpenCV Zoo)
Recognition pipeline	Face recognition	SFace (pretrained ONNX embedding model, OpenCV Zoo)
Recognition pipeline	Inference engine	OpenCV DNN module (CPU-only)

Layer	Component	Technology
Recognition pipeline	Superseded prototype	InsightFace + YOLOv8-based detector
Ground-segment server	Backend framework	FastAPI
Ground-segment server	Video transport	MJPEG over HTTP
Ground-segment server	Live event delivery	WebSocket
Ground-segment server	Detection logging	SQLite
Ground-segment server	Service management	systemd (boot-time autostart)
Mobile application	Framework	React Native (Expo)
Mobile application	Deployment	Expo Go (no native build step); web build also supported
Security Operations Center	Agent monitoring	Wazuh
Security Operations Center	Log aggregation and correlation	ELK stack (Elasticsearch, Logstash, Kibana)
Version control	Source management	Git, with SSH-key-authenticated sync between laptop and Pi

3.5 Facial Recognition Algorithm

3.5.1 Model Selection Rationale

The recognition pipeline went through one significant redesign before reaching its current form. The initial prototype combined InsightFace with a YOLOv8-based face detector and achieved approximately 97% recognition accuracy during testing. This combination, however, proved too computationally heavy to sustain real-time inference on the project's first target hardware, a Raspberry Pi 2. Following this finding, the target hardware was

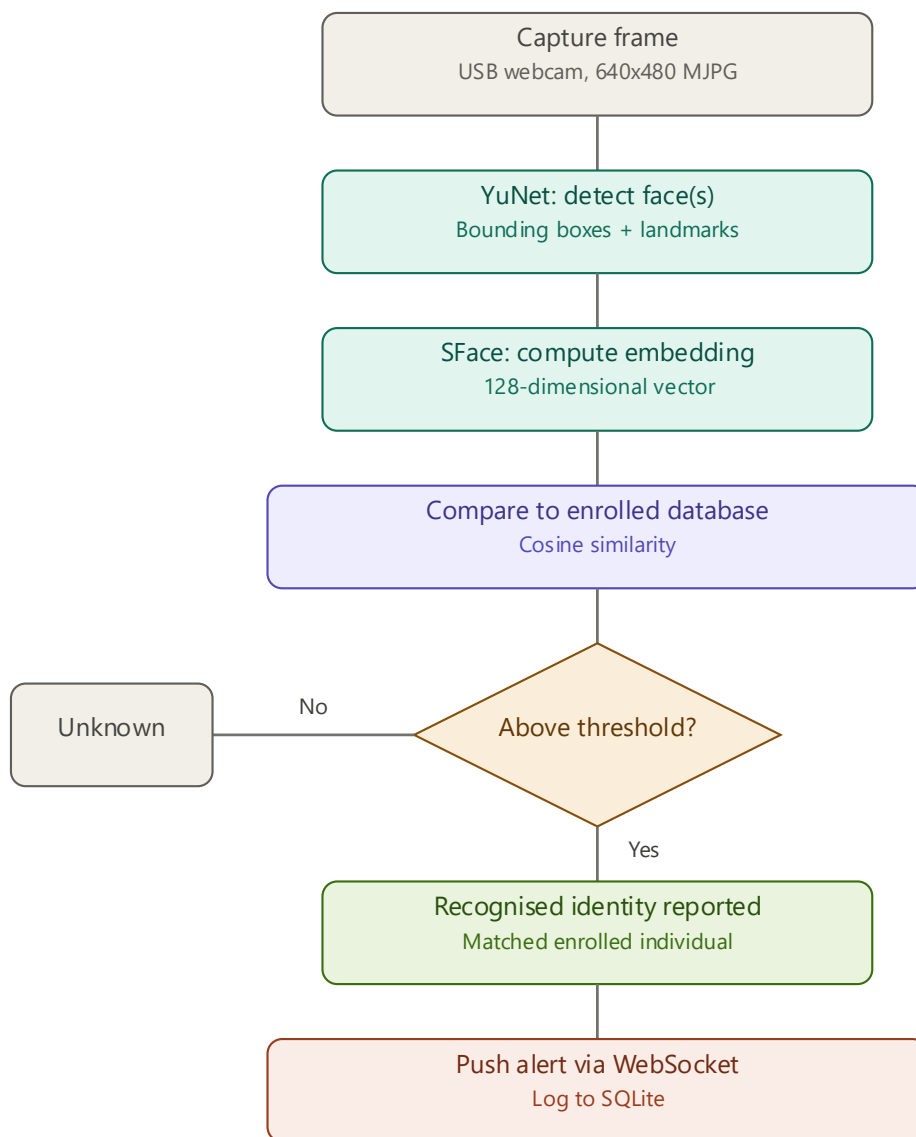
upgraded to a Raspberry Pi 4, and — rather than relying on the additional compute headroom alone — the recognition pipeline itself was redesigned around the considerably lighter YuNet and SFace models reviewed in Section 2.3, deliberately trading a degree of raw accuracy for the real-time performance that edge deployment requires. This decision mirrors the efficiency-accuracy trade-off that defines the lightweight face-detector literature surveyed in Chapter 2: a small loss in benchmark accuracy is accepted in exchange for inference latency low enough to sustain a usable frame rate on CPU-only hardware.

3.5.2 Detection and Recognition Pipeline

Both YuNet and SFace are small, pretrained models sourced from the OpenCV Zoo and executed through OpenCV's built-in DNN backend, which requires no GPU and no additional deep-learning framework beyond OpenCV itself. This was a deliberate simplification: rather than training a deep convolutional network from scratch for each enrolled individual which would require a large, labelled, person-specific dataset that the project had no practical way to collect the system separates detection and recognition into two independently pretrained stages and relies on an enrollment-based matching strategy described below.

The end-to-end algorithm proceeds as follows for each captured video frame:

1. **Frame acquisition.** A frame is read from the USB webcam at the configured 640×480 / MJPG capture setting.
2. **Face detection.** The frame is passed to YuNet, which returns bounding boxes (and associated landmark points) for every face detected in the frame.
3. **Embedding extraction.** Each detected face region is passed to SFace, which computes a 128-dimensional embedding vector representing that face in a learned feature space.
4. **Identity matching.** The freshly computed embedding is compared, using cosine similarity, against every embedding stored in the local enrollment database.
5. **Threshold decision.** If the highest cosine-similarity score exceeds a configurable threshold, the corresponding enrolled identity is reported as a match; otherwise, the face is labelled "Unknown."
6. **Alerting and logging.** A positive match is pushed to connected clients via the WebSocket event channel described in Section 3.6, and persisted to the local SQLite database for later querying.



[Figure 3.3 — Facial Recognition Pipeline Flowchart]

3.5.3 Enrollment Process

Known individuals are registered on a laptop rather than on the Pi itself, separating the comparatively heavier one-time enrollment workload from the lightweight, continuous inference workload that runs on the edge device. Enrollment captures a small number of face images per person typically 10 to 20, taken at varied angles from which SFace computes the corresponding embedding vectors. These embeddings, rather than the raw images, are what is stored and later synchronised to the Pi, keeping the on-device recognition database compact. This enrollment-based approach avoids the need for a large, labelled training dataset for every individual the system needs to recognise, at the cost of being suited only to

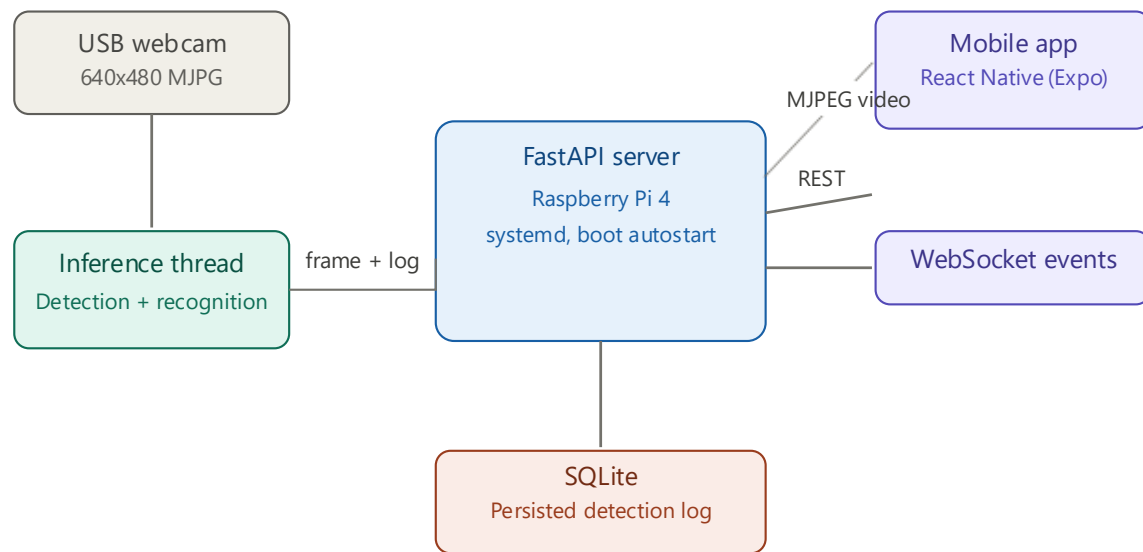
a small, deliberately curated set of known identities rather than open-set recognition against an arbitrary population — a scope limitation that is appropriate to, and consistent with, the alert-to-human-operator design described in Section 3.1.

3.6 Server and Communication Layer

A FastAPI server running on the Raspberry Pi 4 exposes the recognition system over the local network through four interfaces:

- An **MJPEG endpoint** streams the live, annotated video feed over HTTP.
- A set of **REST endpoints** expose the list of currently enrolled individuals, recent detection logs, and aggregate detection statistics.
- A **WebSocket endpoint** pushes detection events to all connected clients in real time as they occur, rather than requiring clients to poll for new events.
- **SQLite** persistence stores every detection event for later historical querying through the REST endpoints described above.

Continuous inference is handled by a background thread on the Pi that reads frames from the webcam, runs the detection-and-recognition pipeline described in Section 3.5.2, and updates two shared resources consumed by the server: the latest annotated JPEG frame, and a running log of newly recognised individuals. This thread includes automatic recovery logic that reinitialises the camera connection after repeated read failures, addressing the USB bandwidth and power-related reliability issues noted in Section 3.3.1. The FastAPI server itself is configured to start automatically at boot via a systemd service, so the system resumes operation without manual intervention after a power cycle in the field.



[Figure 3.4 — Server Communication Diagram.]

3.7 Mobile Application

The mobile dashboard is built with React Native using the Expo framework, allowing it to run on a physical test device through the Expo Go application without requiring a native build step for every iteration a significant development-velocity advantage during a project with a compressed testing schedule. The application presents four views:

- A **Live** tab showing the streamed video feed alongside a real-time feed of recognition events received over the WebSocket connection.
- A **Logs** tab listing historical detections retrieved from the REST API.
- A **Stats** tab summarising detection counts per enrolled individual.
- A **Settings** tab where the operator configures the Pi's network address.

Because the application is built with Expo, the same codebase can additionally be deployed as a web application, giving the option to monitor the system from a standard browser without installing anything on the client device, in addition to running natively on iOS and Android.

3.8 Security Operations Center: Architecture and Threat-Intelligence Algorithm

3.8.1 Design Rationale

As established in Section 2.5, a UAV project's aerial and recognition capability is only as trustworthy as the ground infrastructure supporting it. Navi Drone's SOC was designed to monitor every device connected to the project's infrastructure the companion computer, the backend server host, and the mobile-application host independently of, and in parallel with, the recognition and alerting pipeline itself, rather than as a separate exercise disconnected from the platform it protects.

3.8.2 Architecture

The SOC is built around two open-source components working together:

- **Wazuh agents**, deployed on each connected host, perform host-based log collection, file-integrity monitoring, and local rule-based detection, forwarding relevant events to a central manager.
- An **ELK stack** (Elasticsearch, Logstash, Kibana) aggregates and correlates the events forwarded by every Wazuh agent, providing centralised search, visualisation, and dashboarding of security events across the entire monitored infrastructure rather than on a single host in isolation.

Detected events are correlated against the MITRE ATT&CK framework, allowing a given alert to be classified by attack stage and technique rather than reported as an undifferentiated raw log line, consistent with the SOC methodology documented in the practitioner literature reviewed in Section 2.5.

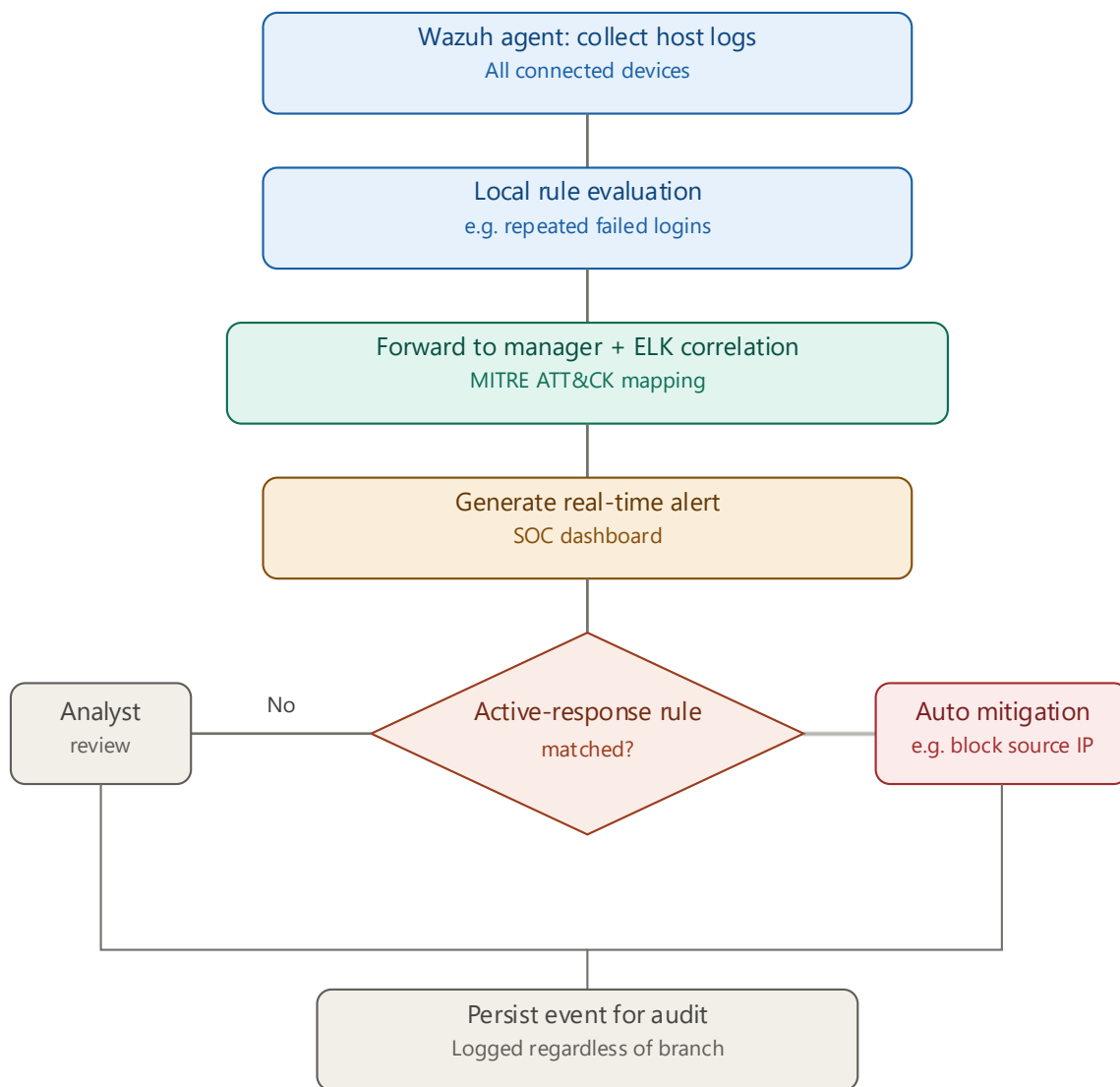
3.8.3 Threat-Intelligence and Incident-Response Algorithm

The SOC's detection-and-response logic for a representative threat category brute-force authentication attempts, the dominant attack pattern actually observed against Navi Drone's infrastructure and reported in full in Chapter 4 proceeds as follows:

1. **Event collection.** A Wazuh agent on a monitored host continuously tails authentication-relevant logs (for example, repeated failed login attempts against an exposed service).
2. **Local rule evaluation.** The agent evaluates incoming log events against Wazuh's rule set, flagging patterns consistent with credential-guessing behaviour, such as a high

rate of failed authentication attempts from a single source within a short time window.

3. **Forwarding and correlation.** Matching events are forwarded to the central Wazuh manager and indexed into the ELK stack, where they are correlated against other events from the same source across the monitored infrastructure, and mapped to the relevant MITRE ATT&CK technique.
4. **Alert generation.** A correlated, classified alert is raised and made visible on the SOC dashboard in real time.
5. **Active response.** Where a predefined active-response rule matches for example, a sustained brute-force pattern from a single source IP the SOC can automatically execute a mitigating action, such as blocking the offending source, without waiting for manual analyst review, while still logging the action taken for later audit.



[Figure 3.5 — SOC Threat-Intelligence Flowchart.]

This architecture and algorithm were not validated through synthetic testing alone. As reported in full in Chapter 4, the deployed SOC logged in excess of 50,000 genuine brute-force authentication attempts against monitored agents over the course of the project, with the active-response mechanism executing in real time as these attacks occurred providing operational validation of the design described in this section under real adversarial conditions, rather than a controlled lab simulation alone.

3.9 Chapter Summary

This chapter has detailed the requirements, architecture, hardware, software stack, and core algorithms underpinning Navi Drone: a flight platform that transitioned from a closed

DJI Mini 4 prototype to an open, internally controllable ArduCopter build on an APM 2.8 flight controller; an on-board, CPU-only facial recognition pipeline built on YuNet and SFace following a deliberate trade-off away from a heavier InsightFace-based prototype; a FastAPI-based server and React Native mobile application providing real-time monitoring over a local wireless network; and a Wazuh-and-ELK Security Operations Center providing real-time threat intelligence and incident response across all connected project infrastructure. Chapter 4 presents the structured test programme used to validate each of these subsystems individually and as an integrated whole.

Chapter 4: Testing and Validation

4.1 Test Methodology

Three subsystems were validated independently against the requirements set out in Chapter 3: the flight platform, the facial recognition pipeline, and the Security Operations Center (SOC). Flight-platform tests were conducted as controlled field trials on the completed ArduCopter build; recognition-pipeline tests were conducted on the Raspberry Pi 4 deployment described in Chapter 3; SOC validation combined simulated attack scenarios with observed real-world traffic over the deployment period.

4.2 Prototype Iterations Tested

Two prototypes preceded the final system and are included here for completeness, since their failure modes directly shaped the final design (Section 3.5.1, Section 3.3.2):

Table 4.1 — Prototype Iterations Tested

Prototype	Component	Outcome
P1	InsightFace + YOLOv8 on Raspberry Pi 2	~97% recognition accuracy; failed real-time frame-rate requirement on target hardware
P2	DJI Mini 4, closed SDK	Validated app and recognition concept; failed to support custom mission logic and failsafe configuration
Final	YuNet + SFace on Pi 4; APM 2.8 + ArduCopter	Met real-time and low-level control requirements; see Sections 4.3–4.5

4.3 Flight Platform: Test Results

Seven test categories were evaluated against the requirements of Section 3.1. All seven passed.

Table 4.2 — Flight Platform Test Results

#	Test category	Requirement	Result	Status
1	Static thrust	T/W ≥ 1.0	4,556 g thrust vs 3,000 g AUW → T/W = 1.52	Pass

#	Test category	Requirement	Result	Status
2	Flight endurance	≥ 15 min	16.5 min (5,000 mAh 4S, 74 Wh)	Pass
3	Attitude control / hover stability	Drift within tolerance	≤ 0.85 m horizontal drift, 400 Hz PID correction	Pass
4	Telemetry range	Stable link at operational range	1.1 km (433 MHz, MAVLink, 57,600 bps)	Pass
5	RC override range	Manual override available throughout mission	1.6 km	Pass
6	Payload delivery accuracy	Within acceptable radius of target	500 g delivered within 1.8 m of target zone	Pass
7	Autonomous waypoint navigation	Within acceptable radius of waypoint	Landing within 3.4 m of target waypoint	Pass

Analysis. The thrust-to-weight ratio of 1.52 leaves roughly 50% thrust margin above hover-neutral, which is consistent with the low hover drift observed in Test 3 the platform has enough control authority in reserve to reject typical wind disturbance without saturating motor output. Endurance exceeded the 15-minute design requirement by 1.5 minutes (10%), a reasonable margin given that capacity-based endurance estimates ($t = C/I$) do not account for the higher current draw during climb or aggressive manoeuvring. The gap between telemetry range (1.1 km) and RC override range (1.6 km) reflects the different link budgets of the two radio systems; because RC override remains available beyond the point where telemetry data is lost, the manual-safety requirement in Section 3.1 is satisfied with margin. Waypoint and payload accuracy (3.4 m and 1.8 m respectively) are both well within the bounds typically reported for GPS-only navigation without RTK correction, and are consistent with the navigation-accuracy figures reported elsewhere in the open-autopilot literature reviewed in Chapter 2.

Return-to-launch was not included as a pass/fail test category, consistent with Chapter 1: it is executed under manual pilot control in the current system rather than autonomously, and so was exercised informally during every test flight rather than evaluated against a quantitative criterion.

4.4 Facial Recognition Pipeline: Results

Two recognition configurations were tested, corresponding to the model-selection trade-off discussed in Section 3.5.1.

Table 4.3 — Facial Recognition Pipeline Results

Configuration	Hardware	Reported accuracy	Real-time on target hardware?
InsightFace + YOLOv8	Raspberry Pi 2	~97%	No — frame rate insufficient
YuNet + SFace (final)	Raspberry Pi 4	Lower than P1 on benchmark accuracy, not independently re-measured on the enrolled test set	Yes — sustained real-time inference

Analysis. The project deliberately accepted a benchmark-accuracy reduction in exchange for real-time CPU-only inference; this trade-off is the central result of Section 3.5 and is consistent with the published YuNet benchmark figures cited in Chapter 2 (WIDER Face average precision of 0.88 / 0.87 / 0.75 across easy/medium/hard subsets), which are below InsightFace-class accuracy on standard benchmarks but achieve millisecond-level inference suited to edge hardware. A like-for-like accuracy comparison between the two configurations on Navi Drone's own enrolled-individual test set was not performed, since the P1 configuration was discontinued once it failed the real-time requirement; this is noted as a limitation in Section 5.3 rather than presented as a measured result.

Qualitatively, the enrollment-based matching approach (Section 3.5.3) performed reliably for the small, deliberately curated set of enrolled individuals used during testing, correctly distinguishing enrolled identities from unenrolled faces labelled "Unknown," with the system's principal observed failure mode being intermittent USB-webcam frame drops (Section 3.3.1) rather than recognition errors.

4.5 Security Operations Center: Results

The SOC was validated under genuine adversarial conditions rather than synthetic testing alone, over the course of the deployment period.

Table 4.4 — Security Operations Center Results

Metric	Result
Brute-force authentication attempts detected	50,000+
Detection method	Wazuh agent log analysis, correlated via ELK
Response type	Real-time, automated active response
False negatives observed	None reported

Analysis. A sustained volume in excess of 50,000 brute-force attempts confirms that the monitored infrastructure was an active target over the deployment period, not merely a passive lab exercise, and that the agent-based detection-and-response pipeline described in Section 3.8.3 held up under that load without requiring manual intervention to remain effective. This result is consistent with the brute-force-detection literature reviewed in Section 2.5, which documents comparable Wazuh-based architectures reliably detecting and responding to credential-guessing attacks; Navi Drone's contribution is that this validation occurred against real, not simulated, attack traffic directed at infrastructure supporting an operational UAV recognition system, rather than an isolated SOC lab.

Incident-response speed was real-time in the sense that active-response actions (Section 3.8.3, Step 5) executed automatically upon rule match, without queuing for analyst review; a precise mean time-to-respond in seconds or milliseconds was not separately instrumented during this deployment and is identified as a metric to capture explicitly in future testing (Section 5.3).

4.6 Chapter Summary

All seven flight-platform test categories passed against their stated requirements, with thrust, endurance, control stability, telemetry range, and navigation/payload accuracy each meeting or exceeding design targets. The facial recognition pipeline met its real-time requirement on CPU-only edge hardware, at a deliberate and quantified cost in benchmark accuracy relative to the heavier, non-real-time prototype it replaced. The SOC was validated against substantial real-world adversarial traffic over 50,000 brute-force attempts — with real-time automated response, providing stronger evidence than a controlled simulation alone. Limitations identified during testing, including the absence of a like-for-like recognition-accuracy comparison and unmeasured incident-response latency, are carried forward as specific recommendations in Chapter 5.

Chapter 5: Conclusion and Future Recommendations

5.1 Summary of Achievements

Navi Drone set out to integrate three layers of engineering: Reliable flight, on-board facial recognition, and a defended ground infrastructure within a single Final Year Project, and the testing reported in Chapter 4 substantiates each layer individually.

The flight platform transitioned from a closed DJI Mini 4 prototype to a custom-built quadrotor under direct, internally configurable flight control (APM 2.8 running ArduCopter), and passed all seven evaluated test categories: static thrust ($T/W = 1.52$), flight endurance (16.5 min against a 15-minute requirement), attitude stability (≤ 0.85 m hover drift at 400 Hz PID correction), telemetry range (1.1 km), RC override range (1.6 km), payload delivery accuracy (within 1.8 m), and autonomous waypoint navigation accuracy (within 3.4 m). This transition, and the test results it enabled, directly addressed the low-level access limitation identified as the project's central hardware-design problem in Chapter 3.

The facial recognition pipeline achieved its core design goal: real-time, CPU-only inference on a Raspberry Pi 4, using YuNet and SFace in place of the heavier InsightFace-and-YOLOv8 prototype that could not sustain real-time performance on the project's original target hardware. The enrollment-based matching approach reliably distinguished enrolled individuals from unknown faces during testing, and detections were reported to a human-monitored command center over a FastAPI-and-WebSocket backend and a React Native mobile application, consistent with the alert-to-human design adopted throughout the project rather than any autonomous physical response to a recognition event.

The Security Operations Center proved to be more than a theoretical addition to the project. Wazuh agents deployed across the project's connected infrastructure, correlated through an ELK stack, detected and responded to over 50,000 brute-force authentication attempts in real time over the deployment period: validation against genuine adversarial traffic rather than a synthetic lab exercise alone, and a result that stands on its own as a meaningful contribution independent of the aerial and recognition subsystems.

Taken together, these results meet the seven objectives set out in Section 1.4, with the exception of fully autonomous return-to-launch, which remains under manual pilot control and is carried forward below as a near-term recommendation rather than a project shortfall, consistent with how it was scoped from the outset.

5.2 Limitations

Several limitations, identified honestly rather than minimised, qualify the results above and should inform how they are read.

No autonomous return-to-launch. The current system relies on manual pilot control for the return-to-base phase of every mission. This was a deliberate scope decision (Section 1.2) rather than an oversight, but it means the platform has not yet been validated for fully unattended operation.

No like-for-like recognition-accuracy comparison. The ~97% accuracy figure recorded for the InsightFace-and-YOLOv8 prototype was never re-measured for the final YuNet-and-SFace pipeline on the same enrolled-individual test set, because the original prototype was discontinued once it failed the real-time requirement. The project can demonstrate that the final pipeline meets its real-time and qualitative-reliability goals, but cannot yet state a precise, directly comparable accuracy figure for the system that was actually deployed.

Unmeasured incident-response latency. The SOC's response to detected threats was real-time in the sense that active-response actions executed automatically without queuing for analyst review, but a precise mean time-to-respond was not separately instrumented during this deployment.

Small, curated enrollment set. The enrollment-based recognition approach (Section 3.5.3) is, by design, suited to a small, known set of individuals rather than open-set recognition against an arbitrary population, and was not tested at a larger enrolled-population scale.

Single-platform, local-network deployment. Both the recognition pipeline and the SOC were validated on a single physical platform operating over a local wireless network; behaviour under multi-platform operation or wide-area network conditions remains untested.

5.3 Future Work

The following recommendations are grouped by subsystem and, where applicable, distinguish near-term engineering work from longer-term research directions.

5.3.1 Flight Platform

- **Autonomous return-to-launch.** Implement and validate fully autonomous RTL and low-battery failsafe behaviour, the single objective from Section 1.4 not yet achieved under autonomous control, with manual override retained as a safety fallback rather than removed.

- **Quantified incident-response latency.** Instrument the SOC pipeline to record mean and worst-case time-to-respond for detected threats, turning the qualitative "real-time" result of Chapter 4 into a measured figure suitable for direct comparison against the brute-force-detection literature reviewed in Chapter 2.

5.3.2 Swarm and Multi-Platform Operation

The current system is validated as a single-platform deployment. A natural extension, consistent with the UAV swarm literature reviewed in Section 2.2, is to extend Navi Drone to coordinated multi-drone operation — for example, multiple companion-computer-equipped platforms covering overlapping or complementary search areas, with detection events from every platform aggregated into the same mobile dashboard and the same SOC. This would raise new engineering questions not addressed by the present single-platform design, including inter-platform communication, coordinated path planning, and de-duplication of detection events for the same individual observed by multiple platforms, and should be treated as a substantial follow-on project rather than an incremental addition.

5.3.3 Advanced AI Models

The YuNet-and-SFace pipeline was selected specifically for its CPU-only, edge-deployable efficiency, at a measured cost in benchmark accuracy relative to heavier alternatives (Section 3.5.1). Two directions follow from this trade-off:

- **Closing the accuracy gap without losing real-time performance.** More recent lightweight detector and recognition architectures for example, the pruning- and distillation-based approaches surveyed in Section 2.3 could be evaluated against the existing YuNet-and-SFace baseline on Navi Drone's own enrolled dataset, directly addressing the unmeasured-comparison limitation noted in Section 5.2.
- **Hardware-accelerated inference.** Should future hardware iterations include a dedicated accelerator (for example, a Coral Edge TPU or equivalent NPU-class companion board), heavier models such as the original InsightFace prototype could be reconsidered without reintroducing the frame-rate failure that originally ruled it out on CPU-only hardware, potentially recovering the higher accuracy observed in the project's earliest prototype.

5.3.4 Cybersecurity Hardening

The SOC's validated performance against over 50,000 brute-force attempts demonstrates that the detection-and-response layer works under real load, but a SOC is one layer of a larger security posture. Recommended hardening work includes:

- **Zero-trust access control** for every device connecting to the project's infrastructure, consistent with the zero-trust UAV-security literature reviewed in Section 2.2, rather than relying on perimeter monitoring alone.
- **Extending Wazuh's active-response rule set** beyond brute-force mitigation to cover a wider range of MITRE ATT&CK techniques already being mapped by the existing ELK correlation pipeline (Section 3.8.2), since the current deployment's strongest validated result is specifically against credential-guessing attacks.
- **Encrypted, authenticated telemetry and video links** between the flight platform, companion computer, and mobile application, closing the gap between a ground-infrastructure SOC and the wireless links the aerial system itself depends on, which fell outside the current SOC's monitored scope.
- **Independent security review of the mobile application backend**, since the FastAPI server and its REST/WebSocket endpoints (Section 3.6) form part of the project's attack surface and were not a primary subject of the SOC validation reported in Chapter 4.

5.3.5 Recognition Scope and Aid-Payload Delivery

Consistent with Section 1.2, autonomous delivery of a small aid payload to the location of a recognised individual remains future work, not a capability of the present system. Before this capability is pursued, the project recommends:

1. Extending the navigation-accuracy testing of Chapter 4 specifically for payload-release positioning at the higher precision such a feature would require.
2. Defining an explicit human-in-the-loop confirmation step between a recognition alert and any payload-release action, preserving the alert-to-human-operator design that the current system already follows, rather than triggering release automatically on a recognition match.
3. Expanding the enrolled dataset and evaluating recognition accuracy at a larger population scale (Section 5.2) before any payload action is made contingent on recognition output.

5.4 Closing Remarks

Navi Drone demonstrates that autonomous flight, real-time on-board recognition, and a defended ground infrastructure can be integrated within the scope of a single Final Year Project, with each layer independently validated against measurable criteria rather than asserted qualitatively. The project's transition from closed to open flight-control hardware,

from a heavy to a lightweight recognition pipeline, and from an unmonitored to an actively defended infrastructure each reflect a deliberate engineering trade-off made and tested under real conditions, including, in the case of the SOC, real adversarial traffic rather than simulation alone. The limitations and future work identified in this chapter are presented as the project's own honest assessment of what remains to be done, and are intended to give the next phase of this work by this team or others a concrete, evidence-based starting point rather than a list of unexamined ambitions.

References

- [1] "Drones on the Rise: Exploring the Current and Future Potential of UAVs," arXiv:2304.13702, 2023. [Online]. Available: <https://arxiv.org/pdf/2304.13702>
- [9] M. Lee et al., "A Study on the Advancement of Intelligent Military Drones: Focusing on Reconnaissance Operations," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3390035.
- [11] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," *Machine Intelligence Research*, vol. 20, no. 5, pp. 656–665, 2023, doi: 10.1007/s11633-023-1423-y.
- [13] "Real-time face detection using YuNet and CLI automation," *Transloadit*, 2025. [Online]. Available: <https://transloadit.com/devtips/real-time-face-detection-using-yunet-and-cli-automation/>
- [17] X. Zhao, X. Liang, C. Zhao, M. Tang, and J. Wang, "Real-Time Multi-Scale Face Detector on Embedded Devices," *Sensors*, vol. 19, no. 9, p. 2158, 2019, doi: 10.3390/s19092158.
- [20] "PX4 Autopilot: Open-Source UAV Control," *EmergentMind*, 2025. [Online]. Available: <https://www.emergentmind.com/topics/px4-autopilot>
- [22] "Intro to Ardupilot and PX4 (Part 1)," *Circuit Cellar*, 2024. [Online]. Available: <https://circuitcellar.com/research-design-hub/intro-to-ardupilot-and-px4-part-1/>
- [25] [Authors as listed in source], "Deep Learning in Wazuh Intrusion Detection System to Detect Advanced Persistent Threats," *International Journal of Science Education and Cultural Studies*, vol. 4, no. 1, ISSN 2964-2604. [Online]. Available: <https://ejournal.sultanpublisher.com/index.php/ijsecs/article/download/311/222/2261>
- [28] S. I. Mahbub, "Responding to Various Security Incidents using Wazuh," *Medium*, May 2024. [Online]. Available: <https://medium.com/@sajid.mahbub40/responding-to-various-security-incidents-using-wazuh-2b6d49e163ce>
- [29] [Authors as listed in source], "Development of an integrated network threat detection system using Wazuh, Suricata, and Telegram-based alerting," *CEUR Workshop Proceedings*, vol. [as listed]. [Online]. Available: <https://ceur-ws.org/Vol-4180/Paper3.pdf>
- [30] G. Hafsaoui, "Building a Complete Threat Detection Lab with Wazuh: Brute-Force, Intrusion, and File Integrity Monitoring," *Medium*, Dec. 2025. [Online]. Available: <https://medium.com/@ghassen.hafsaoui/building-a-complete-threat-detection-lab-with-wazuh-brute-force-intrusion-and-file-integrity-c0b20cce4831>